

Module 4 Guided Lab Session 1: Request Flow & Visibility

Field	Value
Module	M4 ASP.NET Core 10 Fundamentals
Exercises	1 (Middleware Ordering), 1B (Custom Request Logging Middleware)
After this session you can show	Anonymous calls get 401 on a protected TMS route; every response carries X-Correlation-Id ; logs tie request/response to that id

Welcome to the Backend Foundation Sprint

You keep building the **same** Training Management System (TMS) API you will extend it. A previous build's `Program.cs` **runs**, but behaviour on a sensitive route is wrong: you will discover what is wrong by **observing HTTP status and tracing the pipeline**, then fix it. Later you add tracing so failures are tied to a single request.

In M1 you modelled Student, Course, Enrollment, Assessment, and related ideas. In M4 you host those concerns on the web server. **Session 1** is narrow on purpose: make a sensitive read behave like a protected API in HTTP, then add **request logging + correlation ID** so support and debugging are grounded in evidence.

By the end of Exercise 1: GET `/api/assessments/results` rejects anonymous callers with **401** (no assessment JSON for unauthenticated requests). You earn that outcome by fixing how the pipeline is wired—**without** relying on comments inside the starter file to name the mistake for you.

By the end of Exercise 1B: you wrap the pipeline in custom logging, emit **X-Correlation-Id**, and register `ExceptionHandler` early—the same posture you will reconnect when Session 3 adds `ProblemDetails` and `Scalar`.

Before you begin: the `TmsApi` project

Module 4 uses **one** ASP.NET Core Web API project named **TmsApi**. Create it **once** at the start of Session 1 and keep working in that same folder through Session 3 so exercises stack on each other. If you closed your machine and are **continuing M4**, open the existing `TmsApi` directory you already created—do **not** run `dotnet new` again for a second API.

Open your terminal and run these commands **one time** when you begin M4:

```
# Check your SDK version: must show 10.x
dotnet --version
```

Create the Web API project (controller-based template; matches later exercises)

```
dotnet new webapi -n TmsApi --no-openapi --use-controllers
```

Move into the project directory

```
cd TmsApi
```

Open it in VS Code (or Visual Studio)

```
code .
```

Find Program.cs. You will rewrite it throughout this session. Confirm the project builds and runs:

```
dotnet run
```

You should see output like:

```
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5xxx
```

Press Ctrl+C to stop the server.

Troubleshooting

Problem	Likely cause	Fix
dotnet --version shows 8.x or 9.x	Wrong SDK	Install .NET 10 from dot.net
Port already in use	Another process on 5000	dotnet run --urls "http://localhost:5050"
Template produces minimal API not controllers	Default changed in some SDK builds	Always pass --use-controllers as shown above
dotnet: command not found	SDK not on PATH	Reopen terminal after install; or fully qualify the path

Testing HTTP in this session: Session 3 wires **OpenAPI + Scalar**. Until then, use the **browser** for simple GETs (you will see **401** text) and **browser DevTools** → **Network** → **Response headers** to confirm **X-Correlation-Id**. After Scalar is available at /scalar/v1, repeat the same checks with **Try it**—that becomes your default. Use curl only if you need a scripted check or your facilitator asks for it.

Every URL example below uses http://localhost:5000; replace it with the port your terminal prints.

Exercise 1: The Blind Server (Middleware Ordering)

Context: Assessment outcomes must stay confidential. The route GET /api/assessments/results returns a **placeholder** JSON body for now (you will replace it with real persistence and DTOs in later modules). Below is **starter** Program.cs from a prior check-in. Run it, hit the route

anonymously, then read the file **top to bottom** and decide what should happen for a protected read versus what actually happens.

Predict before you change code

[!CAUTION] Trace one incoming GET in order: which app.Use... / Map... lines run, and in what sequence? Where does the response get committed? Write your prediction in one sentence, then run the app and compare.

```
// Starter pipeline (do not assume this order is correct)
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.UseRouting();

app.MapGet("/api/assessments/results", () => Results.Ok(new
{
    courseCode = "CS-101",
    studentId = "S-001",
    letterGrade = "A"
}));

app.UseAuthentication();
app.UseAuthorization();

app.Run();
```

Your task: secure the pipeline

Rewrite Program.cs so GET /api/assessments/results is **not** anonymous: unauthenticated callers get **401**, not the JSON body. You keep the same placeholder response for callers who **are** allowed through (your cohort's auth scheme may be minimal this early—follow your facilitator).

Work from your template's usual Program.cs shape; **do not** paste a full solution from elsewhere unless your facilitator green-lights it. Use this scaffold and fill the TODOs from your understanding of the pipeline:

```
var builder = WebApplication.CreateBuilder(args);
// Services: add authentication / authorization services if your template or facilitator requires them for
this exercise

var app = builder.Build();

// TODO 1: Register routing in the pipeline where it belongs for your app.
// TODO 2: Register authentication and authorization in the pipeline where your template and facilitator
expect them for a protected minimal API route.
// TODO 3: Map GET /api/assessments/results with the same response body as the starter, but require
authorization for that route.
```

app.Run();

Run / verify / common failure

- **Run:** dotnet run
- **Check (preferred before Scalar):** Browser GET <http://localhost:5000/api/assessments/results> with **no** login or bearer token.
- **Expected: 401 Unauthorized** not **200** with JSON, not **404**.
- **After Scalar exists:** Same request via Scalar **Try it**; confirm status **401**.
- **Common failure 404:** Terminal middleware or branch ordering—re-read the middleware / request-pipeline section in **Module 4 ASP.NET Core Fundamentals** (your Essentials reading).
- **Common failure still 200 with body on anonymous GET:** Middleware that should run for this route is not in the active pipeline path—trace order again or ask for a hint in the room.

Exercise 1B: Custom Request Logging Middleware

Context: Order is fixed, but you still cannot tie log lines to a single failing request. Implement middleware that logs start/finish and stamps **X-Correlation-Id**.

Your Task (implement, do not transcribe a full solution)

1. Add RequestLoggingMiddleware.cs with a RequestDelegate next and ILogger<RequestLoggingMiddleware>.
2. In InvokeAsync:
 - Generate a short correlation id (for example from Guid.NewGuid().ToString("N")[..8]).
 - Set context.Response.Headers["X-Correlation-Id"] **before** await next(context).
 - Use Stopwatch to measure elapsed time.
 - Log one line on entry (method, path, correlation id) and one on exit (status code, elapsed ms, same id).
3. In Program.cs, register **in this order** (adjust only if your facilitator's template differs):
 - app.UseMiddleware<RequestLoggingMiddleware>(); first (outer wrapper).
 - Then UseExceptionHandler, UseHttpsRedirection, UseRouting, UseAuthentication, UseAuthorization.
 - Map GET /api/assessments/results last, still with .RequireAuthorization().

If you are unsure where UseExceptionHandler fits, use **Module 4 ASP.NET Core Fundamentals** (middleware and error-handling sections) or your **Session 3** materials—you are wiring the slot now so later exercises can plug in ProblemDetails without reordering everything.

Run / verify / common failure

- **Run:** dotnet run
- **Check:** Same anonymous GET as Exercise 1 (/api/assessments/results).
- **Expected:** Response still **401**; response headers include **X-Correlation-Id**; **two** log lines per request sharing the same correlation id.
- **Common failure no logs:** Middleware registered after the endpoint.
- **Common failure no header:** Header set after await next(context) (too late).

Why this session ends here

You have only done two things—and that is enough for Session 1:

1. **Exercise 1:** A **TMS assessment** route is **not** reachable anonymously (401 without auth); the pipeline matches what you learned in the Essentials reading on request flow.
2. **Exercise 1B:** Every request is **observable**: correlation id in the response and paired log lines. You also **registered** UseExceptionHandler in the stack you will extend when error bodies become ProblemDetails.

You have **not** yet built real assessment queries, Scalar, or full CRUD—that is intentional. Session 2 adds services and configuration discipline; Session 3 connects this pipeline to controllers, OpenAPI, and Scalar.

Session 1 checkpoint

Before Session 2, confirm:

- ☐ Anonymous GET /api/assessments/results returns **401**, not 200 with JSON.
- ☐ Every response includes **X-Correlation-Id** (DevTools Network headers, or Scalar response headers after Session 3).
- ☐ Console shows **two** structured lines per request with the **same** correlation id.
- ☐ Optional: when /scalar/v1 exists, you can repeat the GET in **Try it** and see the same status and header without a curl wall.

If any check fails, finish Session 1 before moving on.